

SHELL = command-line interpreter

SCRIPT = text file containing
Linux commands and shell directives.

REFERENCE SHEET FOR SHELL TEST (LAB)

> send text into file
(output)

>> append text to file
(output)

| wc -l count lines

CTRL+C → kill

CTRL+Z → PAUSE

PID process ID

PPID parent process ID

TTY terminal

TIME CPU time used

CMD command name

ps -f

ps -ef

ps -aux → all processes
w user info

< input redirection

<< append to input

find . -name "*.txt" find recursively all files ending in ".txt" in current dir.
(everyone)

wc count lines, words, bytes

head -n 2 file.txt first two lines

tail -n 2 file.txt last two lines

sort file.txt | uniq remove duplicates

sort file.txt | uniq -c count duplicates

while [\$i -lt 7]; do
... i=\$((i+1))
done

if [\$a -lt \$b]; then
echo t: min = \$a
else
echo e: min = \$b
fi

\$(command) command
substitution
'command'

#!/ SHEBANG → directive
#!/bin/bash → interpreter

• send mail to g13:

grep g13 passwd.fake | cut -d ':' -f 1

grep g13 passwd.fake | cut -d ':' -f 1 > mails.txt

send.sh: \$1 → address of field
"\$1" → value of field

for p in \$(grep g13 passwd.fake | cut -d ':' -f 1); do
echo sending mail to \$p → check if user exists.
sleep 1
done

GREP

• search in files using REGEX.

* → zero or more
+ → one or more
? → zero or one
{2,3} → at least 2

grep "1\$USER:"

grep -E -v "1,2,3\$"

grep "dam" passwd.fake all lines containing dam

grep -i "1dam:" passwd.fake all lines starting with "dam:"

grep -E "1[10-9]:" passwd.fake all users with no digits
in their username.

grep "." file.txt non-empty lines

grep "1\$" file.txt empty lines

" " → whenever a var.
might contain spaces
or is empty

chmod +x file.txt add execute
chmod -x file.txt remove execute
chmod 700 file.txt

K = \$((\$K+1))

m = \$((\$m-1))

INCREMENT DECREMENT

for p; do
: done

not so many
times (slow)

SHELL → interpreted
C → compiled
↳ many times (fast)

1 → start of line
[^] → negation
[a-z] → lowercase
[0-9] → digit
+ → one or more
* → zero or more
() → group
| → or
\ → escape
" " → taken literally
" " → confusion if
regex has \$

SED • eliminates / replaces stuff → search in place.

`echo "Alex" | sed -E 's/e/3/d' → Al3x`

`sed -E "s/[^a-z]//gi"` → everything not a character replace it with nothing.

`sed -E "s/+/\n/g"` → replace all spaces with a newline.

STRING

`x="hello" → echo "$x"`

`x=$(ls)`

ARRAY

`x=(a,b,c) → echo "${x[@]}"`

`x[0], x[1], x[2]`

AWK • extract fields as columns.

`grep -E "grglu" passwd.fake | awk -F: '{print $5}'`

print student's name field.

`$@, $*` → all args as a string/array

`$#` → nr. of args.

`$?` → exit code of previous command.

`*.txt` → all .txt files in the current directory

`x=$(find...)`

↳ STRING!

`()` → ARRAY

`$()` → STRING

• Count all lines in c files of the directory given as param, excluding empty lines or lines containing blank spaces.

`#!/bin/bash`

`s=0`

`for f in $(ls *.c); do`

`m=$(grep "[^t]" $f | wc -l)`

`s=$((s+m))`

`done`

`echo $s`

... FILE INPUT
1) `done < the file` → read input from file:

`while read i; do`

(ex 7)

`done < the file`

2) `done << (find "$1" ...)`

`while IFS= read -r f; do`

(ex 2)

`done << (find "$1" ...)`

← sends contents of "\$allFiles" to the loop as input.

3) `done <<< "$allFiles"` ← VARIABLE INPUT

`while read -r f; do`

`done <<< "$allFiles"`

• `currently comm=$(cat who.fake | cut -d' ' -f 1 | sort | uniq)`

• `nrprocesses=$(grep "^$usr" ps.fake | wc -l)`

• `if [ $# -eq 0 ]; then`

`echo "No args"`

• `while IFS= read -r f; do`

....

`done << (find "$1" -....)`

• `if [-d "$1"]` → check directory

• `if [-e "$1"]` → check existence (file/dir)

• `if [-f "$1"]` → check file

• `if [-r "$1"]` → readable check

• `if [-s "$1"]` → check not-empty

• `if [! -d "$1"]` → not directory.

• `if [-L "$1"]` → check symbolic path.

• `if [! -e "$1"]` → broken path.

• `find "$1" -type f -name "*.c"; find "$directory" -name "*.log"`

• `files=( $(...) )` ARRAY → access with `"${files[@]}"`

`files=$(...)` STRING → access with `"$files"`

`files=$(find "$dir" -type f)`

• `sort "$file" -o "$file"` → sort content of file and write it to the same file.

• `processes=( $(ps -ef | grep "$danger" | awk '{print $2}' | head -n-2) )`

• `if [ $$ != $pr ]` → I am not killing myself.

• `(find "$1" -type f -perm -222)` → everyone (owner + group + others) can write.

• `(stat -c "%A" "$file")` → human readable form.

• `chmod a-w "$file"` → remove write for all users.

• `grep -2 "1$1:" passwd.fake` → take usernames.